

Introduction à l'Intelligence Artificielle (BUT 2) – Programmation Logique

Vincent Risch

`vincent.risch@univ-amu.fr`

- ➊ Introduction : aux origines de la programmation logique...
- ➋ PROLOG
- ➌ ASP (Answer Set Programming)

Introduction : aux origines de la programmation logique...

Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci.

Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :

Programmation...

... Logique

Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :

Programmation...

... Logique

Démonstration Automatique



Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :

Programmation...

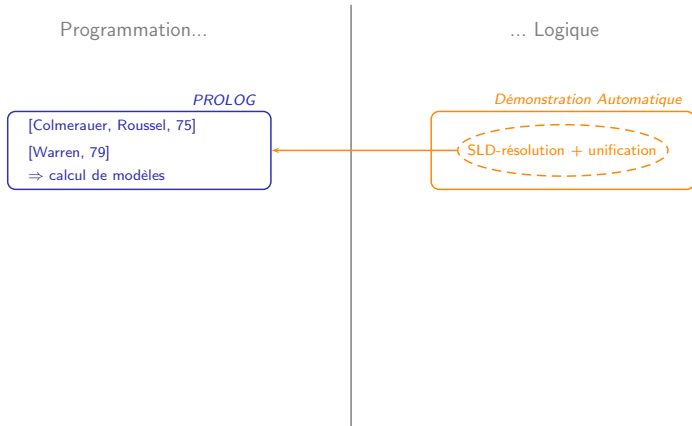
... Logique

Démonstration Automatique

(SLD-résolution + unification)

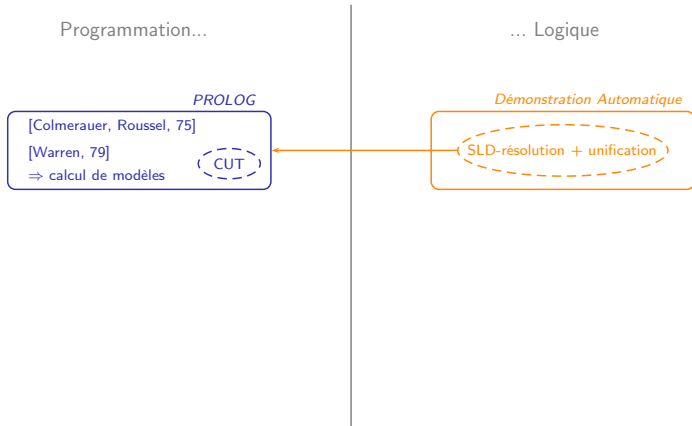
Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :



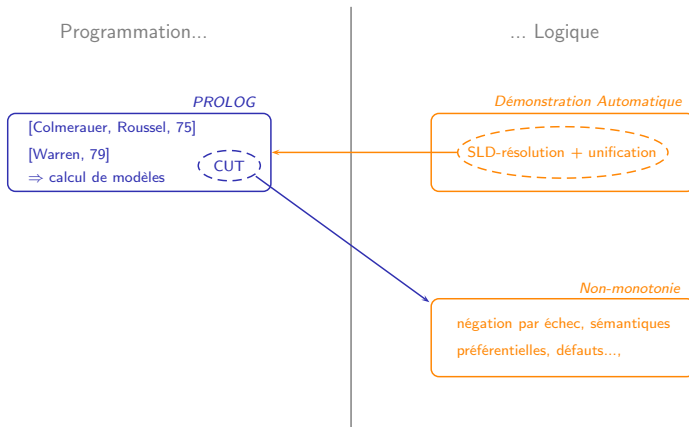
Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :



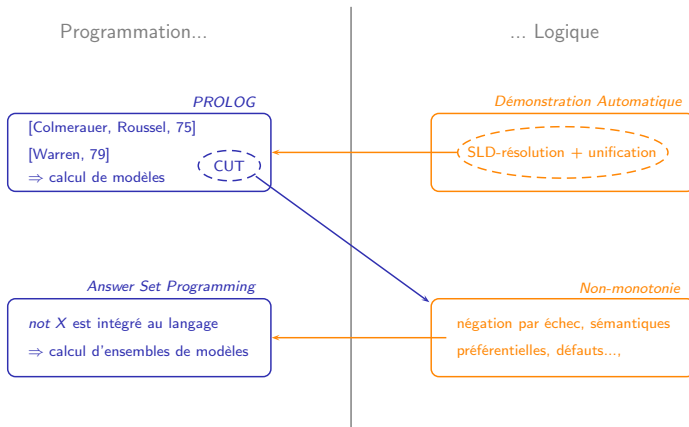
Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :



Introduction

La *Programmation Logique* est un paradigme de programmation permettant de représenter des connaissances symboliques et de calculer sur celles-ci. Historiquement :



Exemple : un menu

Informations de base : relations entre éléments

hors-d'œuvre

Artichauts Mélanie
Truffes sous le sel
Cresson œuf-poché

viande

Grillade de bœuf
Poulet au tilleul

poisson

Bar aux algues
Sole meunière

dessert

Sorbet aux poires
Fraises chantilly
Melon en surprise

Exemple : un menu

Informations de base : relations entre éléments

hors-d'œuvre

Artichauts Mélanie
Truffes sous le sel
Cresson œuf-poché

viande

Grillade de bœuf
Poulet au tilleul

poisson

Bar aux algues
Sole meunière

dessert

Sorbet aux poires
Fraises chantilly
Melon en surprise

Relations = prédicats

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilleul)

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

Exemple : un menu

Informations structurées : connections entre relations

$$\begin{aligned}(\forall p)(\text{viande}(p) \rightarrow \text{plat}(p)), \quad & (\forall p)(\text{poisson}(p) \rightarrow \text{plat}(p)), \\ (\forall e)(\forall p)(\forall d)(\text{hors-d'uvre}(e) \wedge \text{plat}(p) \wedge \text{dessert}(d) \\ & \rightarrow \text{repas}(e, p, d))\end{aligned}$$

Exemple : un menu

Informations structurées : connections entre relations

$$\begin{aligned} &(\forall p)(\text{viande}(p) \rightarrow \text{plat}(p)), & (\forall p)(\text{poisson}(p) \rightarrow \text{plat}(p)), \\ &(\forall e)(\forall p)(\forall d)(\text{hors-d'uvre}(e) \wedge \text{plat}(p) \wedge \text{dessert}(d) \\ & \hspace{15em} \rightarrow \text{repas}(e, p, d)) \end{aligned}$$

Après transformation : un ensemble de clauses de Horn

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilleul)

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \text{viande}(p) \vee \text{plat}(p)$

$\neg \text{poisson}(p) \vee \text{plat}(p)$

$\neg \text{hors-d'œuvre}(e) \vee \neg \text{plat}(p) \vee \neg \text{dessert}(d) \vee \text{repas}(e, p, d)$

Clause de Horn

Une *clause de Horn* est une clause dont au plus un littéral est positif. Une clause de Horn ayant exactement un littéral positif est appelée *clause définie*.

Clause de Horn

Une *clause de Horn* est une clause dont au plus un littéral est positif. Une clause de Horn ayant exactement un littéral positif est appelée *clause définie*.

SLD-resolution [Kowalsky, 73]

Selected Linear Definite clause-resolution : repose

Clause de Horn

Une *clause de Horn* est une clause dont au plus un littéral est positif. Une clause de Horn ayant exactement un littéral positif est appelée *clause définie*.

SLD-resolution [Kowalsky, 73]

Selected Linear Definite clause-resolution : repose

- sur l'emploi de l'unique règle d'inférence

$$\frac{\neg P \vee \neg Q_1 \vee \neg Q_2 \vee \dots \vee Q_n \quad P \vee \neg Q'_1 \vee \dots \vee \neg Q'_m}{\neg Q_1 \vee \neg Q_2 \vee \dots \vee Q_n \vee \neg Q'_1 \vee \dots \vee \neg Q'_m}$$

Clause de Horn

Une *clause de Horn* est une clause dont au plus un littéral est positif. Une clause de Horn ayant exactement un littéral positif est appelée *clause définie*.

SLD-resolution [Kowalsky, 73]

Selected Linear Definite clause-resolution : repose

- sur l'emploi de l'unique règle d'inférence

$$\frac{\neg P \vee \neg Q_1 \vee \neg Q_2 \vee \dots \vee Q_n \quad P \vee \neg Q'_1 \vee \dots \vee \neg Q'_m}{\neg Q_1 \vee \neg Q_2 \vee \dots \vee Q_n \vee \neg Q'_1 \vee \dots \vee \neg Q'_m}$$

- sur le calcul d'un plus grand *unificateur* à chaque étape ;

Clause de Horn

Une *clause de Horn* est une clause dont au plus un littéral est positif. Une clause de Horn ayant exactement un littéral positif est appelée *clause définie*.

SLD-resolution [Kowalsky, 73]

Selected Linear Definite clause-resolution : repose

- sur l'emploi de l'unique règle d'inférence

$$\frac{\neg P \vee \neg Q_1 \vee \neg Q_2 \vee \dots \vee Q_n \quad P \vee \neg Q'_1 \vee \dots \vee \neg Q'_m}{\neg Q_1 \vee \neg Q_2 \vee \dots \vee Q_n \vee \neg Q'_1 \vee \dots \vee \neg Q'_m}$$

- sur le calcul d'un plus grand *unificateur* à chaque étape ;
- sur le *Th. d'Herbrand* : une conjonction de littéraux est valide ssi la clause vide est dérivable par réduction à l'absurde.

Unification [Robinson, 65]

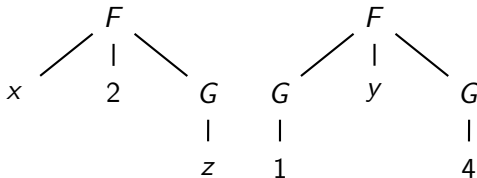
→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

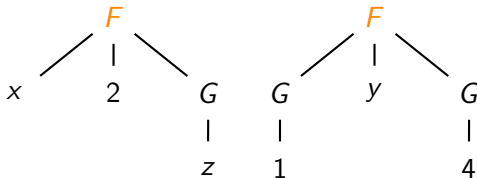
$$F(x, 2, G(z)) = F(G(1), y, G(4))$$



Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

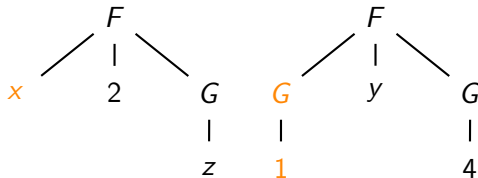
$$F(x, 2, G(z)) = F(G(1), y, G(4))$$



Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

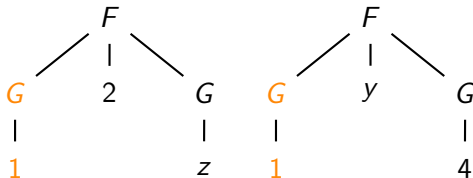
$$F(x, 2, G(z)) = F(G(1), y, G(4))$$



Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$

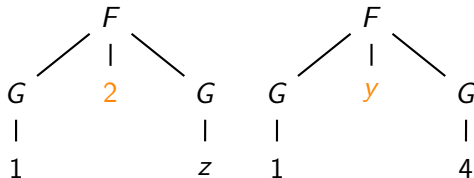


$$(x = G(1))$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$

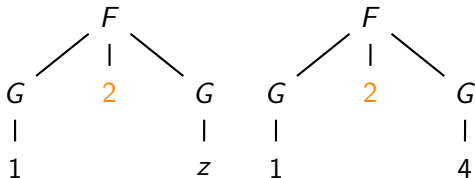


$$(x = G(1))$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$

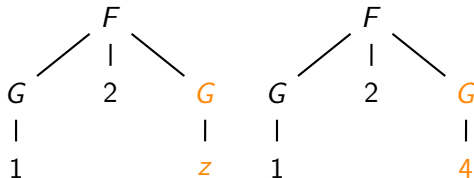


$$(x = G(1))(y = 2)$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$



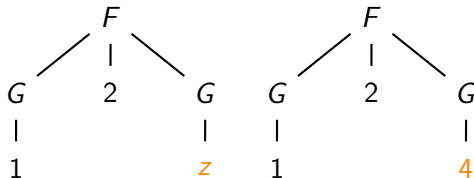
$$(x = G(1))(y = 2)(G(z) = G(4))$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(\textcolor{brown}{z})) = F(G(1), y, G(\textcolor{brown}{4}))$$



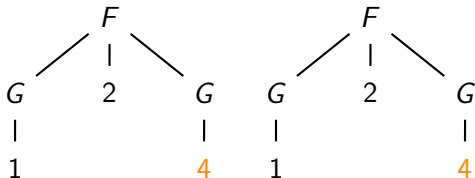


$$(x = G(1))(y = 2)(\textcolor{brown}{G(z) = G(4)})$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$

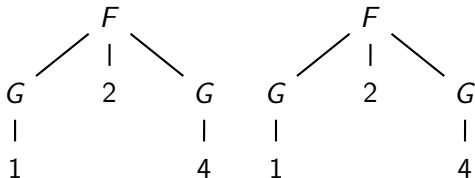


$$(x = G(1))(y = 2)(z = 4)$$

Unification [Robinson, 65]

→ Propos : comment rendre égaux deux termes ?

$$F(x, 2, G(z)) = F(G(1), y, G(4))$$



$$(x = G(1))(y = 2)(z = 4)$$

Un algorithme d'unification

```
PlusGrandUnifieur fonction Unification (Terme  $T1$ ,  $T2$ )  
   $Subst = \{\}$ ;  
  Tant que ( $Subst\ T1 \neq Subst\ T2$ ) faire {  
    Soient  $t1$ ,  $t2$  dans  $T1$ ,  $T2$  tels que  $t1 \neq t2$ ;  
    Si (aucun n'est une variable ou  
      l'un est une variable contenue dans l'autre)  
    alors  $T1$  et  $T2$  ne sont pas unifiables;  
    sinon {  
      Soit  $x$  une variable parmi  $T1$ ,  $T2$ ;  
      Soit  $t$  celui de  $T1$ ,  $T2$  qui n'est pas dans  $x$ ;  
       $Subst = Subst.(x=t)$ ;  
    }  
  }  
  retourner  $Subst$ ;
```

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilléul)

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

$\neg \textit{poisson}(p) \vee \textit{plat}(p)$

$\neg \textit{hors-d'œuvre}(e) \vee \neg \textit{plat}(p) \vee \neg \textit{dessert}(d) \vee \textit{repas}(e, p, d)$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilléul)

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

$\neg \textit{poisson}(p) \vee \textit{plat}(p)$

$\neg \textit{hors-d'œuvre}(e) \vee \neg \textit{plat}(p) \vee \neg \textit{dessert}(d) \vee \textit{repas}(e, p, d)$

$\neg \textit{repas}(\text{Artichauts Mélanie}, \text{Bar aux algues}, \text{Sorbet aux poires})$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilléul)

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

$\neg \textit{poisson}(p) \vee \textit{plat}(p)$

$\neg \textit{hors-d'œuvre}(e) \vee \neg \textit{plat}(p) \vee \neg \textit{dessert}(d) \vee \textit{repas}(e, p, d)$

$\neg \textit{repas}(\text{Artichauts Mélanie}, \text{Bar aux algues}, \text{Sorbet aux poires})$



$(e = \text{Artichauts Mélanie})(p = \text{Bar aux algues})(d = \text{Sorbet aux poires})$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilleul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

$\neg \textit{hors-d'œuvre}(\textit{Artichauts Mélanie}) \vee$

$\neg \textit{plat}(\textit{Bar aux algues}) \vee$

$\neg \textit{dessert}(\textit{Sorbet aux poires})$

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \textit{poisson}(p) \vee \textit{plat}(p)$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Artichauts Mélanie)

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilléul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

$\neg \textit{hors-d'œuvre}(\textit{Artichauts Mélanie}) \vee$

$\neg \textit{plat}(\textit{Bar aux algues}) \vee$

$\neg \textit{dessert}(\textit{Sorbet aux poires})$

poisson(Bar aux algues)

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \textit{poisson}(p) \vee \textit{plat}(p)$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)
hors-d'œuvre(Cresson œuf-poché)
viande(Grillade de bœuf)
viande(Poulet au tilléul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

$\neg \textit{plat}(\text{Bar aux algues}) \vee$
 $\neg \textit{dessert}(\text{Sorbet aux poires})$

poisson(Bar aux algues)
poisson(Sole meunière)
dessert(Sorbet aux poires)
dessert(Fraises chantilly)
dessert(Melon en surprise)

$\neg \textit{poisson}(p) \vee \textit{plat}(p)$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)
hors-d'œuvre(Cresson œuf-poché)
viande(Grillade de bœuf)
viande(Poulet au tilloul)

$\neg \text{viande}(p) \vee \text{plat}(p)$

$\neg \text{plat}(\text{Bar aux algues}) \vee$
 $\neg \text{dessert}(\text{Sorbet aux poires})$

($p = \text{Bar aux algues}$)

poisson(Bar aux algues)
poisson(Sole meunière)
dessert(Sorbet aux poires)
dessert(Fraises chantilly)
dessert(Melon en surprise)

$\neg \text{poisson}(p) \vee \text{plat}(p)$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)
hors-d'œuvre(Cresson œuf-poché)
viande(Grillade de bœuf)
viande(Poulet au tilleul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

poisson(Bar aux algues)
poisson(Sole meunière)
dessert(Sorbet aux poires)
dessert(Fraises chantilly)
dessert(Melon en surprise)

$\neg \textit{poisson}(\text{Bar aux algues}) \vee$
 $\neg \textit{dessert}(\text{Sorbet aux poires})$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)
hors-d'œuvre(Cresson œuf-poché)
viande(Grillade de bœuf)
viande(Poulet au tilleul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

poisson(Bar aux algues)
poisson(Sole meunière)
dessert(Sorbet aux poires)
dessert(Fraises chantilly)
dessert(Melon en surprise)
 $\neg \textit{poisson}(\text{Bar aux algues}) \vee$
 $\neg \textit{dessert}(\text{Sorbet aux poires})$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilleul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

poisson(Sole meunière)

dessert(Sorbet aux poires)

dessert(Fraises chantilly)

dessert(Melon en surprise)

$\neg \textit{dessert}(\text{Sorbet aux poires})$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)
hors-d'œuvre(Cresson œuf-poché)
viande(Grillade de bœuf)
viande(Poulet au tilleul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

poisson(Sole meunière)
dessert(Sorbet aux poires)
dessert(Fraises chantilly)
dessert(Melon en surprise)
 $\neg \textit{dessert}(\textit{Sorbet aux poires})$

repas(Artichauts Mélanie, Bar aux algues, Sorbet aux poires) ?

hors-d'œuvre(Truffes sous le sel)

hors-d'œuvre(Cresson œuf-poché)

viande(Grillade de bœuf)

viande(Poulet au tilleul)

$\neg \textit{viande}(p) \vee \textit{plat}(p)$

poisson(Sole meunière)

dessert(Fraises chantilly)

dessert(Melon en surprise)

}

Ensemble des clauses

- | | |
|--|---|
| [C ₀] <i>hors-d'œuvre</i> (Artichauts Mélanie) | [C ₅] <i>poisson</i> (Bar aux algues) |
| [C ₁] <i>hors-d'œuvre</i> (Truffes sous le sel) | [C ₆] <i>poisson</i> (Sole meunière) |
| [C ₂] <i>hors-d'œuvre</i> (Cresson œuf-poché) | [C ₇] <i>dessert</i> (Sorbet aux poires) |
| [C ₃] <i>viande</i> (Grillade de bœuf) | [C ₈] <i>dessert</i> (Fraises chantilly) |
| [C ₄] <i>viande</i> (Poulet au tilleul) | [C ₉] <i>dessert</i> (Melon en surprise) |
| [C ₁₀] $\neg$ <i>viande</i> (<i>p</i>) $\vee$ <i>plat</i> (<i>p</i>) | [C ₁₁] $\neg$ <i>poisson</i> (<i>p</i>) $\vee$ <i>plat</i> (<i>p</i>) |
| [C ₁₂] $\neg$ <i>hors-d'œuvre</i> (<i>e</i>) $\vee$ $\neg$ <i>plat</i> (<i>p</i>) $\vee$ $\neg$ <i>dessert</i> (<i>d</i>) $\vee$ <i>repas</i> (<i>e</i> , <i>p</i> , <i>d</i>) | |

[?] $\neg$ *repas*(Artichauts Mélanie, Bar aux algues, Sorbet aux poires)

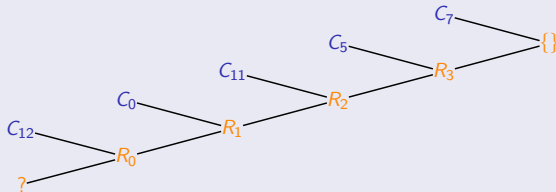
Arbre de dérivation SLD

Ensemble des clauses

- | | |
|--|---|
| $[C_0]$ <i>hors-d'œuvre</i> (Artichauts Mélanie) | $[C_5]$ <i>poisson</i> (Bar aux algues) |
| $[C_1]$ <i>hors-d'œuvre</i> (Truffes sous le sel) | $[C_6]$ <i>poisson</i> (Sole meunière) |
| $[C_2]$ <i>hors-d'œuvre</i> (Cresson œuf-poché) | $[C_7]$ <i>dessert</i> (Sorbet aux poires) |
| $[C_3]$ <i>viande</i> (Grillade de bœuf) | $[C_8]$ <i>dessert</i> (Fraises chantilly) |
| $[C_4]$ <i>viande</i> (Poulet au tilleul) | $[C_9]$ <i>dessert</i> (Melon en surprise) |
| $[C_{10}]$ $\neg$ <i>viande</i> (p) $\vee$ <i>plat</i> (p) | $[C_{11}]$ $\neg$ <i>poisson</i> (p) $\vee$ <i>plat</i> (p) |
| $[C_{12}]$ $\neg$ <i>hors-d'œuvre</i> (e) $\vee$ $\neg$ <i>plat</i> (p) $\vee$ $\neg$ <i>dessert</i> (d) $\vee$ <i>repas</i> (e, p, d) | |

$[?] \neg$ *repas*(Artichauts Mélanie, Bar aux algues, Sorbet aux poires)

Stratégie « linéaire »



PROLOG

Un *programme logique* est un ensemble fini de règles de la forme

$$L_0 \leftarrow L_1 \dots L_m \text{ not } L_{m+1} \dots \text{not } L_n$$

où $L_0, L_1 \dots L_m, L_{m+1} \dots L_n$ sont des littéraux.

Un *programme logique* est un ensemble fini de règles de la forme

$$L_0 \leftarrow L_1 \dots L_m \text{ not } L_{m+1} \dots \text{not } L_n$$

où $L_0, L_1 \dots L_m, L_{m+1} \dots L_n$ sont des littéraux.

En programmation logique classique :

- il n'y a que des littéraux positifs ;
- la liste $\text{not } L_{m+1} \dots \text{not } L_n$ est vide ;
- $L_0 \leftarrow L_1 \dots L_m$ est interprétée comme une *définition* de L_0 , et associée à la clause définie $L_0 \vee \neg L_1 \vee \dots \vee \neg L_m$;
- $L_0 \leftarrow$ est interprétée comme un *fait*.

En PROLOG

- La règle $p(x) \leftarrow q(y) r(z)$ est codée par :

Syntaxe « Marseille »

$p(x) \rightarrow q(y) r(z);$

Syntaxe « Edinburgh »

$p(X) \text{ :- } q(Y), r(Z).$

En PROLOG

- La règle $p(x) \leftarrow q(y) r(z)$ est codée par :

Syntaxe « Marseille »

$p(x) \rightarrow q(y) r(z);$

Syntaxe « Edinburgh »

$p(X) :- q(Y), r(Z).$

- Le fait $p(x) \leftarrow$ est codé par :

Syntaxe « Marseille »

$p(x) \rightarrow;$

Syntaxe « Edinburgh »

$p(X).$

En PROLOG

- La règle $p(x) \leftarrow q(y) r(z)$ est codée par :

Syntaxe « Marseille »

$p(x) \rightarrow q(y) r(z);$

Syntaxe « Edinburgh »

$p(X) :- q(Y), r(Z).$

- Le fait $p(x) \leftarrow$ est codé par :

Syntaxe « Marseille »

$p(x) \rightarrow;$

Syntaxe « Edinburgh »

$p(X).$

→ La syntaxe « Marseille » établit un lien avec les *grammaires de métamorphose*.

→ La syntaxe « Edinburgh » met l'accent sur *l'effacement* des têtes de règles vues comme *buts*.

Un menu en PROLOG

hors_d_oeuvre(artichauts_melanie).
hors_d_oeuvre(truffes_sous_le_sel).
hors_d_oeuvre(cresson_œuf_poche).
viande(grillade_de_bœuf).
viande(poulet_au_tilleul).

poisson(bar_aux_algues).
poisson(sole_meuniere).
dessert(sorbet_aux_paires).
dessert(fraises_chantilly).
dessert(melon_en_surprise).

plat(P) :- *viande*(P).

plat(P) :- *poisson*(P).

repas(E, P, D) :- *hors_d_oeuvre*(E), *plat*(P), *dessert*(D).

Un menu en PROLOG

hors_d_oeuvre(artichauts_melanie).
hors_d_oeuvre(truffes_sous_le_sel).
hors_d_oeuvre(cresson_œuf_poche).
viande(grillade_de_bœuf).
viande(poulet_au_tilleul).

poisson(bar_aux_algues).
poisson(sole_meuniere).
dessert(sorbet_aux_paires).
dessert(fraises_chantilly).
dessert(melon_en_surprise).

plat(P) :- *viande*(P).

plat(P) :- *poisson*(P).

repas(E, P, D) :- *hors_d_oeuvre*(E), *plat*(P), *dessert*(D).

? *repas*(artichauts_melanie, bar_aux_algues, sorbet_aux_paires).

Un menu en PROLOG

hors_d_oeuvre(artichauts_melanie).
hors_d_oeuvre(truffes_sous_le_sel).
hors_d_oeuvre(cresson_œuf_poche).
viande(grillade_de_bœuf).
viande(poulet_au_tilleul).

poisson(bar_aux_algues).
poisson(sole_meuniere).
dessert(sorbet_aux_poires).
dessert(fraises_chantilly).
dessert(melon_en_surprise).

plat(P) :- *viande*(P).

plat(P) :- *poisson*(P).

repas(E, P, D) :- *hors_d_oeuvre*(E), *plat*(P), *dessert*(D).

? *repas*(artichauts_melanie, bar_aux_algues, sorbet_aux_poires).

> Ok

Un menu en PROLOG

hors_d_oeuvre(artichauts_melanie).
hors_d_oeuvre(truffes_sous_le_sel).
hors_d_oeuvre(cresson_œuf_poche).
viande(grillade_de_bœuf).
viande(poulet_au_tilleul).

poisson(bar_aux_algues).
poisson(sole_meuniere).
dessert(sorbet_aux_paires).
dessert(fraises_chantilly).
dessert(melon_en_surprise).

plat(P) :- *viande*(P).

plat(P) :- *poisson*(P).

repas(E, P, D) :- *hors_d_oeuvre*(E), *plat*(P), *dessert*(D).

? *repas*(artichauts_melanie, bar_aux_algues, Z).

Un menu en PROLOG

hors_d_oeuvre(artichauts_melanie).
hors_d_oeuvre(truffes_sous_le_sel).
hors_d_oeuvre(cresson_œuf_poche).
viande(grillade_de_bœuf).
viande(poulet_au_tilleul).

poisson(bar_aux_algues).
poisson(sole_meuniere).
dessert(sorbet_aux_paires).
dessert(fraises_chantilly).
dessert(melon_en_surprise).

plat(P) :- *viande*(P).

plat(P) :- *poisson*(P).

repas(E, P, D) :- *hors_d_oeuvre*(E), *plat*(P), *dessert*(D).

? *repas*(artichauts_melanie, bar_aux_algues, Z).

> Z = sorbet_aux_paires

> Z = fraises_chantilly

> Z = melon_en_surprise

Ce qui fait de PROLOG un langage...

- structures de données faiblement typées : t-uples et listes

Ce qui fait de PROLOG un langage...

- structures de données faiblement typées : t-uples et listes
- l'unification est étendue à des équation/inéquations sur les arbres ;

Ce qui fait de PROLOG un langage...

- structures de données faiblement typées : t-uples et listes
- l'unification est étendue à des équation/inéquations sur les arbres ;
- *cut* rend l'exploration déterministe et fait sortir PROLOG du champs de la démonstration automatique.

Il s'agit d'un prédicat spécifique, noté « / » (Marseille) ou « ! » (Edinburgh), et qui permet de supprimer des buts en attente.

Exemple

```
> ? plat(P), !  
> P = grillade_de_boeuf
```

- Toutes les solutions qui « suivent » sont supprimées
- Ce qui revient à éliminer des modèles...

A un instant donné, soit $[b_1, \dots, b_m]b_{m+1}, \dots, b_n$ avec

- $[b_1, \dots, b_m]$ une liste de buts effacés en attente de remontée ;
- $b_{m+1}, \dots, b_n$ une liste de buts restants à effacer.

A un instant donné, soit $[b_1, \dots, b_m] b_{m+1}, \dots, b_n$ avec

- $[b_1, \dots, b_m]$ une liste de buts effacés en attente de remontée ;
- $b_{m+1}, \dots, b_n$ une liste de buts restants à effacer.

Pour effacer b_{m+1} , on utilise la règle

$$p :- q_1, \dots, q_i, !, q_{i+1} \dots$$

Après unification ($b_{m+1} = p$), la liste des buts évolue de façon suivante :

A un instant donné, soit $[b_1, \dots, b_m] b_{m+1}, \dots, b_n$ avec

- $[b_1, \dots, b_m]$ une liste de buts effacés en attente de remontée ;
- $b_{m+1}, \dots, b_n$ une liste de buts restants à effacer.

Pour effacer b_{m+1} , on utilise la règle

$$p :- q_1, \dots, q_i, !, q_{i+1} \dots$$

Après unification ($b_{m+1} = p$), la liste des buts évolue de façon suivante :

$$[b_1, \dots, b_m, b_{m+1}] \ q_1, \dots, q_i, !, q_{i+1} \dots, b_n$$

A un instant donné, soit $[b_1, \dots, b_m] b_{m+1}, \dots, b_n$ avec

- $[b_1, \dots, b_m]$ une liste de buts effacés en attente de remontée ;
- $b_{m+1}, \dots, b_n$ une liste de buts restants à effacer.

Pour effacer b_{m+1} , on utilise la règle

$$p \text{ :- } q_1, \dots, q_i, !, q_{i+1} \dots$$

Après unification ($b_{m+1} = p$), la liste des buts évolue de façon suivante :

$$\begin{aligned} & [b_1, \dots, b_m, b_{m+1}] \text{ } q_1, \dots, q_i, !, q_{i+1} \dots, b_n \\ & [b_1, \dots, b_m, b_{m+1}, q_1] \text{ } \dots, q_i, !, q_{i+1} \dots, b_n \end{aligned}$$

A un instant donné, soit $[b_1, \dots, b_m] b_{m+1}, \dots, b_n$ avec

- $[b_1, \dots, b_m]$ une liste de buts effacés en attente de remontée ;
- $b_{m+1}, \dots, b_n$ une liste de buts restants à effacer.

Pour effacer b_{m+1} , on utilise la règle

$$p :- q_1, \dots, q_i, !, q_{i+1} \dots$$

Après unification ($b_{m+1} = p$), la liste des buts évolue de façon suivante :

$$[b_1, \dots, b_m, b_{m+1}] \quad q_1, \dots, q_i, !, q_{i+1} \dots, b_n$$

$$[b_1, \dots, b_m, b_{m+1}, q_1] \quad \dots, q_i, !, q_{i+1} \dots, b_n$$

$$[b_1, \dots, b_m, b_{m+1}, q_1, \dots, q_i] \quad !, q_{i+1} \dots, b_n$$

A un instant donné, soit $[b_1, \dots, b_m] b_{m+1}, \dots, b_n$ avec

- $[b_1, \dots, b_m]$ une liste de buts effacés en attente de remontée ;
- $b_{m+1}, \dots, b_n$ une liste de buts restants à effacer.

Pour effacer b_{m+1} , on utilise la règle

$$p :- q_1, \dots, q_i, !, q_{i+1} \dots$$

Après unification ($b_{m+1} = p$), la liste des buts évolue de façon suivante :

$$\begin{aligned} & [b_1, \dots, b_m, b_{m+1}] \quad q_1, \dots, q_i, !, q_{i+1} \dots, b_n \\ & [b_1, \dots, b_m, b_{m+1}, q_1] \quad \dots, q_i, !, q_{i+1} \dots, b_n \\ & [b_1, \dots, b_m, b_{m+1}, q_1, \dots, q_i] \quad !, q_{i+1} \dots, b_n \\ & [b_1, \dots, b_m] \quad q_{i+1} \dots, b_n \end{aligned}$$

Expression de la négation

Menu léger sans « ! »

Soit le programme

`dessert(poire,leger).`

`dessert(X,lourd).`

La requête `? dessert(poire, lourd)` renvoie `Ok`.

Expression de la négation

Menu léger sans « ! »

Soit le programme

```
dessert(poire,leger).
```

```
dessert(X,lourd).
```

La requête ? `dessert(poire, lourd)` renvoie **Ok**.

Menu léger avec « ! »

Soit le programme

```
dessert(poire,leger), !.
```

```
dessert(X,lourd).
```

Cette fois la requête ? `dessert(poire, lourd)` renvoie bien un **échec**.

Expression de la négation

Menu léger sans « ! »

Soit le programme

```
dessert(poire,leger).
```

```
dessert(X,lourd).
```

La requête `? dessert(poire, lourd)` renvoie **Ok**.

Menu léger avec « ! »

Soit le programme

```
dessert(poire,leger), !.
```

```
dessert(X,lourd).
```

Cette fois la requête `? dessert(poire, lourd)` renvoie bien un **échec**.

→ « ! » permet de préciser que *tout ce qui n'est pas répertorié « léger » ne l'est pas...*

Négation par échec

not(P) :- P, !, echec ;

not(P).

où *echec* est un but absent.

Expression de la négation

Négation par échec

not(P) :- P, !, echec ;

not(P).

où *echec* est un but absent.

Propriété

not(toto) s'efface ssi toto ne s'efface pas.

Négation par échec

not(P) :- P, !, echec ;

not(P).

où *echec* est un but absent.

Propriété

not(toto) s'efface ssi toto ne s'efface pas.

En effet :

- si *toto* est effaçable, ! élimine le but *not*(toto) en attente, donc échec, *not*(toto) n'est pas effaçable ;
- si *toto* n'est pas effaçable, ! n'est pas atteint et *not*(toto) est effacé.

Comportement de la négation par échec (1)

Propriétés

not(not(toto)) s'efface ssi *toto* s'efface.

Comportement de la négation par échec (1)

Propriétés

not(not(toto)) s'efface ssi *toto* s'efface.

En effet (cf. résultat précédent) :

- si *toto* s'efface, alors *not(toto)* ne s'efface pas, et donc *not(not(toto))* est effaçable ;
- si *toto* ne s'efface pas, alors *not(toto)* s'efface, et donc *not(not(toto))* ne s'efface pas.

Comportement de la négation par échec (2)

Propriété

Cut rend l'exécution sensible à l'ordre d'évaluation des buts.

Comportement de la négation par échec (2)

Propriété

Cut rend l'exécution sensible à l'ordre d'évaluation des buts.

Conséquences :

- L'ordre d'écriture des règles devient pertinent

Comportement de la négation par échec (2)

Propriété

Cut rend l'exécution sensible à l'ordre d'évaluation des buts.

Conséquences :

- L'ordre d'écriture des règles devient pertinent
- Plus grave : l'ordre des requêtes aussi

aime(pierre, bar_aux_algues).

aime(pierre, poulet_au_tilleul).

poisson(bar_aux_algues).

? *aime*(pierre, X), *not*(*poisson*(X)).

renvoie *X* = poulet_au_tilleul, mais

? *not*(*poisson*(X)), *aime*(pierre, X). renvoie un échec...

Comportement de la négation par échec (2)

Propriété

Cut rend l'exécution sensible à l'ordre d'évaluation des buts.

Conséquences :

- L'ordre d'écriture des règles devient pertinent
- Plus grave : l'ordre des requêtes aussi

```
aime(pierre, bar_aux_algues).  
aime(pierre, poulet_au_tilleul).  
poisson(bar_aux_algues).
```

```
? aime(pierre, X), not(poisson(X)).
```

renvoie `X = poulet_au_tilleul`, mais

```
? not(poisson(X)), aime(pierre, X). renvoie un échec...
```

Conclusion : En PROLOG, la négation par échec s'apparente à un bricolage astucieux.

La négation par échec :

- *n'est pas* la négation logique ;

La négation par échec :

- *n'est pas* la négation logique ;
- exprimerait l'*absence d'un littéral* dans les « conséquences » d'un programme logique ;

La négation par échec :

- *n'est pas* la négation logique ;
- exprimerait l'*absence d'un littéral* dans les « conséquences » d'un programme logique ;
- au delà, exprimerait une forme d'*incompatibilité entre littéraux* ;

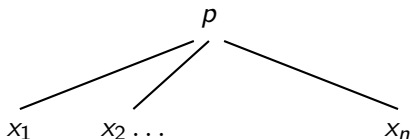
La négation par échec :

- *n'est pas* la négation logique ;
- exprimerait l'*absence d'un littéral* dans les « conséquences » d'un programme logique ;
- au delà, exprimerait une forme d'*incompatibilité entre littéraux* ;
- est (imparfaitement) simulée par l'*élimination de modèles* que permet le *cut*.

Détour : Prolog avec contraintes

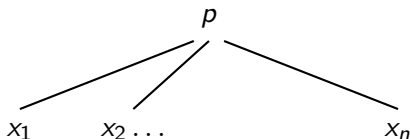
Equations et inéquations sur les arbres

Idée : un prédicat Prolog $p(x_1, x_2, \dots, x_n)$ est identifié à l'arbre



Equations et inéquations sur les arbres

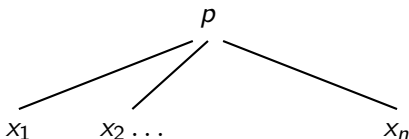
Idée : un prédicat Prolog $p(x_1, x_2, \dots, x_n)$ est identifié à l'arbre



- Rechercher les solutions d'un programme, revient donc à résoudre un ensemble d'*équations* sur des arbres
- Les solutions correspondent à l'ensemble des *affectations* « *sylvestres* » répondant à la question

Equations et inéquations sur les arbres

Idée : un prédicat Prolog $p(x_1, x_2, \dots, x_n)$ est identifié à l'arbre



- Rechercher les solutions d'un programme, revient donc à résoudre un ensemble d'*équations* sur des arbres
- Les solutions correspondent à l'ensemble des *affectations* « *sylvestres* » répondant à la question
- Pourquoi ne pas étendre cette approche à des *inéquations*?

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

- *petite-somme*(U,V,W) exprime simultanément $U + V = W$ et $W < 10$
- repose sur le fait que $U + V = W \Rightarrow (U + 1) + V = W + 1$

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

- *petite-somme*(U,V,W) exprime simultanément $U + V = W$ et $W < 10$
- repose sur le fait que $U + V = W \Rightarrow (U + 1) + V = W + 1$
- à toute question de la forme ? *petite-somme*(X,Y,Z) correspond un ensemble d'affectations sylvestres adéquates

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(4,3,X)

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(4,3,X)

> X=7

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(4,X,7)

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(4,X,7)

> X=3

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(X,Y,5)

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(X,Y,5)

> X=1 Y=4

> X=2 Y=3

> X=3 Y=2

> X=4 Y=1

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(9,1,X)

Exemple : équation sur des entiers

petite-somme

successeur(1, 2).

successeur(2, 3).

successeur(3, 4).

successeur(4, 5).

successeur(5, 6).

successeur(6, 7).

successeur(7, 8).

successeur(8, 9).

petite-somme(1, X, Y) :- *successeur*(X, Y).

petite-somme(X', Y, Z') :- *successeur*(X, X') *petite-somme*(X, Y, Z) *successeur*(Z, Z').

? *petite-somme*(9,1,X)

> pas de solution

Repas léger

```
points(artichauts_melanie, 1).  
points(truffes_sous_le_sel, 2).  
points(cresson_œuf_poche, 3).  
points(grillade_de_bœuf, 7).  
points(poulet_au_tilleul, 6).
```

```
points(bar_aux_algues, 5).  
points(sole_meuniere, 4).  
points(sorbet_aux_paires, 3).  
points(fraises_chantilly, 4).  
points(melon_en_surprise, 2).
```

```
repas-leger(E, P, D) :- repas(E, P, D)  
                        points(E, X) points(P, Y) petite-somme(X, Y, U)  
                        points(D, Z) petite-somme(Z, U, V);
```

Retour sur notre repas

Repas léger

```
points(artichauts_melanie, 1).  
points(truffes_sous_le_sel, 2).  
points(cresson_œuf_poche, 3).  
points(grillade_de_bœuf, 7).  
points(poulet_au_tilleul, 6).
```

```
points(bar_aux_algues, 5).  
points(sole_meuniere, 4).  
points(sorbet_aux_piores, 3).  
points(fraises_chantilly, 4).  
points(melon_en_surprise, 2).
```

```
repas-leger(E, P, D) :- repas(E, P, D)  
                        points(E, X) points(P, Y) petite-somme(X, Y, U)  
                        points(D, Z) petite-somme(Z, U, V);
```

petite-somme

```
successeur(1, 2).  
successeur(2, 3).  
successeur(3, 4).  
successeur(4, 5).
```

```
successeur(5, 6).  
successeur(6, 7).  
successeur(7, 8).  
successeur(8, 9).
```

```
petite-somme(1, X, Y) :- successeur(X, Y).  
petite-somme(X', Y, Z') :- successeur(X, X') petite-somme(X, Y, Z) successeur(Z, Z').
```

Repas léger

```
points(artichauts_melanie, 1).  
points(truffes_sous_le_sel, 2).  
points(cresson_œuf_poche, 3).  
points(grillade_de_bœuf, 7).  
points(poulet_au_tilleul, 6).
```

```
points(bar_aux_algues, 5).  
points(sole_meuniere, 4).  
points(sorbet_aux_piores, 3).  
points(fraises_chantilly, 4).  
points(melon_en_surprise, 1).
```

```
repas-leger(E, P, D) :- repas(E, P, D)  
                        points(E, X) points(P, Y) petite-somme(X, Y, U)  
                        points(D, Z) petite-somme(Z, U, V);
```

? repas-leger(H,P,D)

Repas léger

```
points(artichauts_melanie, 1).  
points(truffles_sous_le_sel, 2).  
points(cresson_œuf_poche, 3).  
points(grillade_de_bœuf, 7).  
points(poulet_au_tilleul, 6).
```

```
points(bar_aux_algues, 5).  
points(sole_meuniere, 4).  
points(sorbet_aux_piores, 3).  
points(fraises_chantilly, 4).  
points(melon_en_surprise, 1).
```

```
repas-leger(E, P, D) :- repas(E, P, D)  
                        points(E, X) points(P, Y) petite-somme(X, Y, U)  
                        points(D, Z) petite-somme(Z, U, V);
```

? repas-leger(H,P,D)

- > H=artichauts_melanie, P=grillade_de_bœuf, D=melon_en_surprise
- > H=truffles_sous_le_sel, P=poulet_au_tilleul, D=melon_en_surprise
- > H=artichauts_melanie, P=poulet_au_tilleul, D=melon_en_surprise
- > H=truffles_sous_le_sel, P=bar_aux_algues, D=melon_en_surprise
- ...

Repas léger

```
points(artichauts_melanie, 1).  
points(truffles_sous_le_sel, 2).  
points(cresson_œuf_poche, 3).  
points(grillade_de_bœuf, 7).  
points(poulet_au_tilleul, 6).
```

```
points(bar_aux_algues, 5).  
points(sole_meuniere, 4).  
points(sorbet_aux_piores, 3).  
points(fraises_chantilly, 4).  
points(melon_en_surprise, 1).
```

```
repas-leger(E, P, D) :- repas(E, P, D)  
                        points(E, X) points(P, Y) petite-somme(X, Y, U)  
                        points(D, Z) petite-somme(Z, U, V);
```

Notre programme énumère toutes les solutions possibles sous la *contrainte* d'une accumulation calorique strictement inférieure à 10

Repas léger

```
repas(E, P, D) :-  
    hors_d_oeuvre(E, U)  
    plat(P, V)  
    dessert(D, W), {  $U = 0, V = 0, W = 0, U + V + W < 10$  } ;
```

Sous réserve qu'initialement $U = 0, V = 0, W = 0$ et que $U + V + W < 10$, E, P, D constituent un repas léger si E un hors-d'œuvre de valeur calorique U , P est un plat de valeur calorique V , D est un dessert de valeur calorique W

ASP (Answer Set Programming) : une approche intégrée de la
négation

Un *programme logique* est un ensemble fini de règles de la forme

$$L_0 \leftarrow L_1 \dots L_m \text{ not } L_{m+1} \dots \text{not } L_n$$

où $L_0, L_1 \dots L_m, L_{m+1} \dots L_n$ sont des littéraux.

Un *programme logique* est un ensemble fini de règles de la forme

$$L_0 \leftarrow L_1 \dots L_m \text{ not } L_{m+1} \dots \text{not } L_n$$

où $L_0, L_1 \dots L_m, L_{m+1} \dots L_n$ sont des littéraux.

En ASP :

- la liste $\text{not } L_{m+1} \dots \text{not } L_n$ n'est pas forcément vide ;
- la règle $L_0 \leftarrow L_1 \dots L_m \text{ not } L_{m+1} \dots \text{not } L_n$ est interprétée intuitivement par « si $L_1 \dots L_m$ sont valides et $\text{not } L_{m+1} \dots \text{not } L_n$ ne sont pas présents alors dériver L_0 » ;
- $L_0 \leftarrow$ correspond à $L_0 \leftarrow \top$;
- $\leftarrow L_1 \dots L_m$ correspond à $\perp \leftarrow L_1 \dots L_m$.

Exemple

Un programme $\mathcal{P} \dots$

$p \leftarrow$ (1)

$q \leftarrow p, \text{ not } r$ (2)

$r \leftarrow p, \text{ not } q$ (3)

Exemple

Un programme $\mathcal{P}...$

$$p \leftarrow \quad (1)$$

$$q \leftarrow p, \text{ not } r \quad (2)$$

$$r \leftarrow p, \text{ not } q \quad (3)$$

- (1 et 2) : p et q sont dérivables ensemble ;

Exemple

Un programme $\mathcal{P} \dots$

$$p \leftarrow \quad (1)$$

$$q \leftarrow p, \text{ not } r \quad (2)$$

$$r \leftarrow p, \text{ not } q \quad (3)$$

- (1 et 2) : p et q sont dérivables ensemble ;
- (1 et 3) : p et r sont dérivables ensemble ;

Un programme $\mathcal{P}...$

$$p \leftarrow \quad (1)$$

$$q \leftarrow p, \text{ not } r \quad (2)$$

$$r \leftarrow p, \text{ not } q \quad (3)$$

- (1 et 2) : p et q sont dérivables ensemble ;
- (1 et 3) : p et r sont dérivables ensemble ;
- (2 et 3) : q et r (avec p) ne sont pas dérivables ensemble.

Exemple

$p \leftarrow$

$q \leftarrow p, \text{ not } r$

$r \leftarrow p, \text{ not } q$

Exemple

$p \leftarrow$

$q \leftarrow p, \text{ ~~not } r~~$

$r \leftarrow p, \text{ not } q$

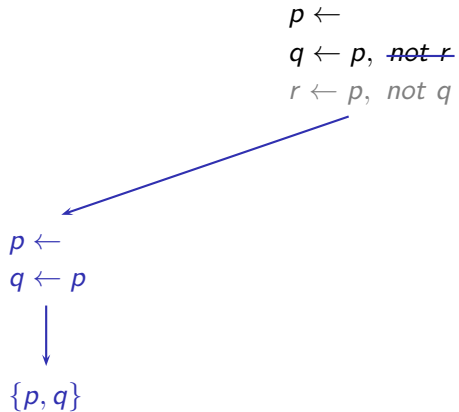
Exemple

$p \leftarrow$
 $q \leftarrow p, \text{ ~~not } r~~$
 $r \leftarrow p, \text{ not } q$

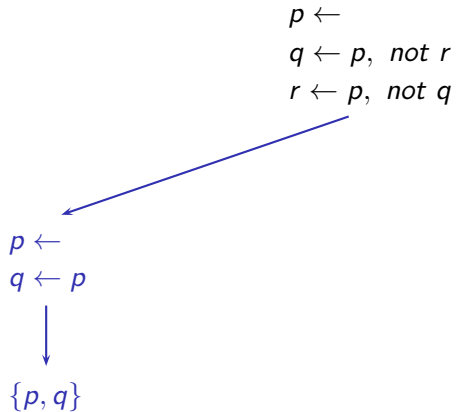


$p \leftarrow$
 $q \leftarrow p$

Example



Example

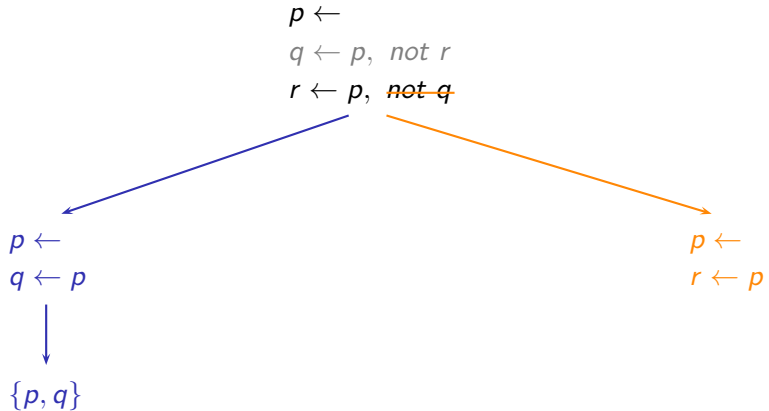


Example

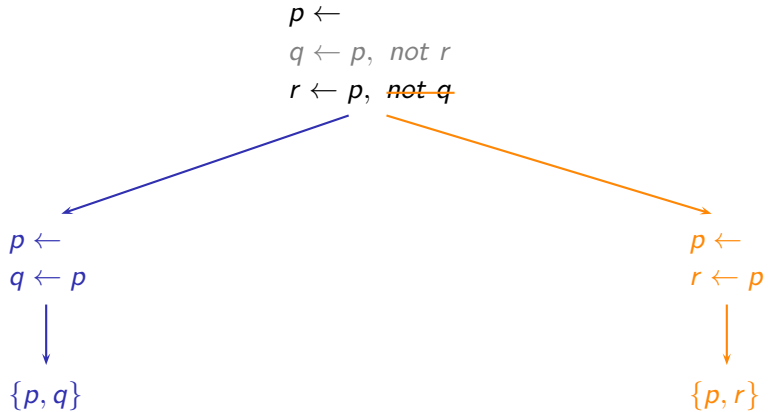
$p \leftarrow$
 $q \leftarrow p, \text{ not } r$
 $r \leftarrow p, \text{ not } q$

$p \leftarrow$
 $q \leftarrow p$
 $\downarrow$
 $\{p, q\}$

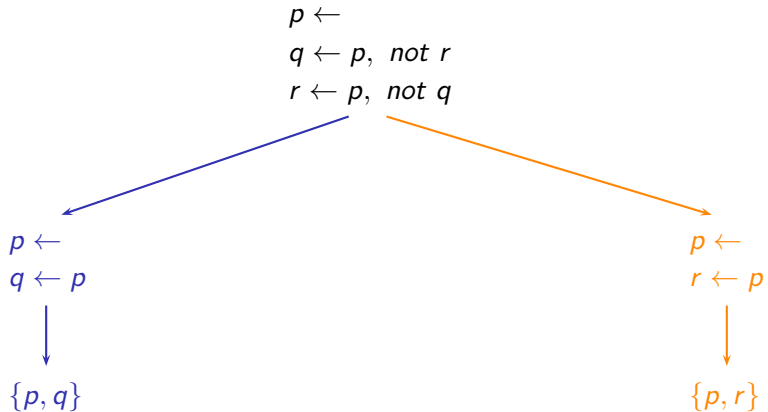
Example



Exemple



Exemple



Etant donnée la règle

$$r = L_0 \leftarrow L_1 \dots L_m \text{ not } L_{m+1} \dots \text{not } L_n$$

on définit :

$$\begin{aligned} \text{head}(r) &= \{L_0\} \\ \text{body}^+(r) &= \{L_1, \dots, L_m\} \\ \text{body}^-(r) &= \{L_{m+1}, \dots, L_n\} \\ \text{body}(r) &= \text{body}^+(r) \cup \text{body}^-(r) \end{aligned}$$

Un programme logique $\mathcal{P}$ est *pur* si $\forall r \in \mathcal{P}, \text{body}^-(r) = \emptyset$.

Un programme logique $\mathcal{P}$ est *pur* si $\forall r \in \mathcal{P}, \text{body}^-(r) = \emptyset$.

Etant donné $\mathcal{P}$, un programme logique pur,

- X un ensemble de littéraux, X est *clos* pour $\mathcal{P}$ si pour chaque règle r de $\mathcal{P}$, $\text{head}(r) \in X$ si $\text{body}(r) \in X$;
- l'ensemble des *conséquences* de $\mathcal{P}$, noté $\text{Cn}(\mathcal{P})$, est le plus petit ensemble de littéraux logiquement clos, et clos pour $\mathcal{P}$.

Etant donné $\mathcal{P}$ un programme logique, X un ensemble de littéraux, on appelle *réduit de $\mathcal{P}$ relativement à X* l'ensemble

$$\mathcal{P}^X = \{ \text{head}(r) \leftarrow \text{body}^+(r) \mid r \in \mathcal{P}, \text{body}^-(r) \cap X = \emptyset \}$$

Etant donné $\mathcal{P}$ un programme logique, X un ensemble de littéraux, on appelle *réduit de $\mathcal{P}$ relativement à X* l'ensemble

$$\mathcal{P}^X = \{head(r) \leftarrow body^+(r) \mid r \in \mathcal{P}, body^-(r) \cap X = \emptyset\}$$

→ $\mathcal{P}^X$ est le programme pur obtenu à partir de $\mathcal{P}$

- en supprimant chaque règle r telle que $body^-(r) \cap X \neq \emptyset$;
- en supprimant *not* $body^-(r)$ des règles restantes.

Etant donné $\mathcal{P}$ un programme logique, X un ensemble de littéraux, on appelle *réduit de $\mathcal{P}$ relativement à X* l'ensemble

$$\mathcal{P}^X = \{head(r) \leftarrow body^+(r) \mid r \in \mathcal{P}, body^-(r) \cap X = \emptyset\}$$

→ $\mathcal{P}^X$ est le programme pur obtenu à partir de $\mathcal{P}$

- en supprimant chaque règle r telle que $body^-(r) \cap X \neq \emptyset$;
- en supprimant *not* $body^-(r)$ des règles restantes.

X est un *answer set* (ou *modèle stable*) de $\mathcal{P}$ ssi $Cn(\mathcal{P}^X) = X$.

Exemple

Un programme $\mathcal{P} \dots$

$$p \leftarrow$$
$$q \leftarrow p, \text{ not } r$$
$$r \leftarrow p, \text{ not } q$$

Exemple

Un programme $\mathcal{P} \dots$

$$p \leftarrow$$
$$q \leftarrow p, \text{ not } r$$
$$r \leftarrow p, \text{ not } q$$

- $X = \{p, q, r\}$. $Cn(\mathcal{P}^X) = \{p\}$, $X \neq Cn(\mathcal{P}^X)$

Exemple

Un programme $\mathcal{P} \dots$

$$p \leftarrow$$
$$q \leftarrow p, \text{ not } r$$
$$r \leftarrow p, \text{ not } q$$

- $X = \{p, q, r\}$. $Cn(\mathcal{P}^X) = \{p\}$, $X \neq Cn(\mathcal{P}^X)$
- $X = \{p, q\}$. $Cn(\mathcal{P}^X) = \{p, q\}$ est un *modèle stable*

Exemple

Un programme $\mathcal{P} \dots$

$$p \leftarrow$$
$$q \leftarrow p, \text{ not } r$$
$$r \leftarrow p, \text{ not } q$$

- $X = \{p, q, r\}$. $Cn(\mathcal{P}^X) = \{p\}$, $X \neq Cn(\mathcal{P}^X)$
- $X = \{p, q\}$. $Cn(\mathcal{P}^X) = \{p, q\}$ est un *modèle stable*
- $X = \{p, r\}$. $Cn(\mathcal{P}^X) = \{p, r\}$ est un *modèle stable*

Un programme $\mathcal{P} \dots$

$$p \leftarrow$$
$$q \leftarrow p, \text{ not } r$$
$$r \leftarrow p, \text{ not } q$$

- $X = \{p, q, r\}$. $Cn(\mathcal{P}^X) = \{p\}$, $X \neq Cn(\mathcal{P}^X)$
- $X = \{p, q\}$. $Cn(\mathcal{P}^X) = \{p, q\}$ est un *modèle stable*
- $X = \{p, r\}$. $Cn(\mathcal{P}^X) = \{p, r\}$ est un *modèle stable*
- $X = \emptyset$. $Cn(\mathcal{P}^X) = \{p, q, r\}$, $X \neq Cn(\mathcal{P}^X)$

Un programme $\mathcal{P} \dots$

$$p \leftarrow \text{not } q$$
$$q \leftarrow \text{not } p$$

Un programme $\mathcal{P}...$

$$p \leftarrow \text{not } q$$
$$q \leftarrow \text{not } p$$

- PROLOG *ne termine pas* sur la requête p ou la requête q ;

Un programme $\mathcal{P} \dots$

$$p \leftarrow \text{not } q$$
$$q \leftarrow \text{not } p$$

- PROLOG *ne termine pas* sur la requête p ou la requête q ;
- ASP renvoie deux modèles stables : $\{p\}, \{q\}$.

Un programme $\mathcal{P}...$

$$p \leftarrow q$$
$$p \leftarrow r$$
$$q \leftarrow \text{not } r$$
$$r \leftarrow \text{not } q$$

Un programme $\mathcal{P} \dots$

$$p \leftarrow q$$
$$p \leftarrow r$$
$$q \leftarrow \text{not } r$$
$$r \leftarrow \text{not } q$$

- En PROLOG, p *n'est pas dérivable*;

Un programme $\mathcal{P} \dots$

$$p \leftarrow q$$
$$p \leftarrow r$$
$$q \leftarrow \text{not } r$$
$$r \leftarrow \text{not } q$$

- En PROLOG, p *n'est pas dérivable*;
- ASP renvoie deux modèles stables : $\{p, q\}, \{p, r\}$.

ASP résulte d'une vision différente de la programmation logique :

- l'accent est mis sur les *modèles* (et non sur les preuves associées aux substitutions sur des termes) comme les solutions d'un problème ;

ASP résulte d'une vision différente de la programmation logique :

- l'accent est mis sur les *modèles* (et non sur les preuves associées aux substitutions sur des termes) comme les solutions d'un problème ;
- on y dispose d'une *sémantique* alternative pour la négation par échec ;

ASP résulte d'une vision différente de la programmation logique :

- l'accent est mis sur les *modèles* (et non sur les preuves associées aux substitutions sur des termes) comme les solutions d'un problème ;
- on y dispose d'une *sémantique* alternative pour la négation par échec ;
- du coup le codage y est *purement déclaratif*.

Minimalité

Tout modèle stable M de $\mathcal{P}$ est un modèle minimal de $\mathcal{P}$ (au sens de $\subseteq$).

Exemples : $\mathcal{P} = \{q \leftarrow p\}, M = \{q\}$ $\mathcal{P} = \{p \leftarrow\}, M = \{p\}$

Minimalité

Tout modèle stable M de $\mathcal{P}$ est un modèle minimal de $\mathcal{P}$ (au sens de $\subseteq$).

Exemples : $\mathcal{P} = \{q \leftarrow p\}, M = \{q\}$ $\mathcal{P} = \{p \leftarrow\}, M = \{p\}$

Non-cumulativité

Si p est dans tout modèle stable M de $\mathcal{P}$, il n'en découle pas que $\mathcal{P}$ et $\mathcal{P} \cup \{p \leftarrow\}$ ont les modèles stables.

Minimalité

Tout modèle stable M de $\mathcal{P}$ est un modèle minimal de $\mathcal{P}$ (au sens de $\subseteq$).

Exemples : $\mathcal{P} = \{q \leftarrow p\}, M = \{q\}$ $\mathcal{P} = \{p \leftarrow\}, M = \{p\}$

Non-cumulativité

Si p est dans tout modèle stable M de $\mathcal{P}$, il n'en découle pas que $\mathcal{P}$ et $\mathcal{P} \cup \{p \leftarrow\}$ ont les modèles stables.

Programme inconsistant

Le program $\{p \leftarrow \text{not } p\}$ n'a pas de modèle stable.

Remarque :

- L'ajout de la règle $p \leftarrow \text{not } p$ à $\mathcal{P}$ « tue » tout modèle stable de $\mathcal{P}$;

Remarque :

- L'ajout de la règle $p \leftarrow \text{not } p$ à $\mathcal{P}$ « tue » tout modèle stable de $\mathcal{P}$;
- L'ajout de $p \leftarrow q_1, \dots, q_m \text{ not } r_1, \dots, \text{not } r_n, \text{not } p$ à $\mathcal{P}$ « tue » tous les modèles stables de $\mathcal{P}$ qui

Remarque :

- L'ajout de la règle $p \leftarrow \text{not } p$ à $\mathcal{P}$ « tue » tout modèle stable de $\mathcal{P}$;
- L'ajout de $p \leftarrow q_1, \dots, q_m \text{ not } r_1, \dots, \text{not } r_n, \text{not } p$ à $\mathcal{P}$ « tue » tous les modèles stables de $\mathcal{P}$ qui
 - contiennent $q_1, \dots, q_m$, et

Remarque :

- L'ajout de la règle $p \leftarrow \text{not } p$ à $\mathcal{P}$ « tue » tout modèle stable de $\mathcal{P}$;
- L'ajout de $p \leftarrow q_1, \dots, q_m \text{ not } r_1, \dots, \text{not } r_n, \text{ not } p$ à $\mathcal{P}$ « tue » tous les modèles stables de $\mathcal{P}$ qui
 - contiennent $q_1, \dots, q_m$, et
 - ne contiennent pas $r_1, \dots, r_n$.

Remarque :

- L'ajout de la règle $p \leftarrow \text{not } p$ à $\mathcal{P}$ « tue » tout modèle stable de $\mathcal{P}$;
- L'ajout de $p \leftarrow q_1, \dots, q_m \text{ not } r_1, \dots, \text{not } r_n, \text{not } p$ à $\mathcal{P}$ « tue » tous les modèles stables de $\mathcal{P}$ qui
 - contiennent $q_1, \dots, q_m$, et
 - ne contiennent pas $r_1, \dots, r_n$.

Intérêt :

Filtrage des modèles stables

On appelle *contrainte d'intégrité* toute règle de la forme

$$\leftarrow q_1, \dots, q_m \text{ not } r_1, \dots, \text{not } r_n$$

Décider si un programme logique $\mathcal{P}$ a un modèle stable est

- NP-complet dans le cas propositionnel ;
- NEXPTIME-complet au premier ordre sans symboles fonctionnels ;
- Σ_1^1 -complet au premier ordre.

Systèmes : CLASP, SMODELS, DLV, ASPERIX ...

Applications : Nombreuses (configuration, routage, puzzles, bio-informatique, robotique, cryptographie,...)